

Cahier de TP

« Confluent Developer Skills for Apache Kafka® »

Pré-requis :

- Bonne connexion Internet
- Système d'exploitation recommandé : Linux, 32Go RAM
- JDK21+, Maven
- IDE Recommandés : STS 4, IntelliJIDEA, VSCode
- Docker, Git
- Apache JMeter
- Python3 et pip

Images utilisées :

- [confluentinc/cp-server:7.7.8](#)
- [docker.redpanda.com/redpandadata/console:v2.3.8](#)
- [confluentinc/cp-schema-registry:7.7.8](#)
- [postgres:15.1](#)
- [dpage/pgadmin4](#)
- [docker.elastic.co/elasticsearch/elasticsearch:8.13.0](#)
- [docker.elastic.co/kibana/kibana:8.13.0](#)
- [quay.io/keycloak/keycloak:26.1](#)

•

Table des matières

Atelier 1: Le cluster kafka.....	3
1.1 Premier démarrage et logs.....	3
1.2 Utilisation des utilitaires.....	3
1.3 Outils graphiques.....	3
Atelier 2: Producer API (Java).....	4
Atelier 2B : Producer via Confluent REST Proxy.....	5
2B.1 Démarrage de la stack.....	5
2B.2 Découverte du REST Proxy.....	5
2B.3 Prise en main du script JMeter.....	6
2B.4 Expérimentation des paramètres de fiabilité (acks).....	7
2B.5 Expérimentation du batching (batch.size et linger.ms).....	9
2B.6 Arrêt d'un borker durant ke test.....	10
Atelier 3 : Consumer API.....	11
3.0 Mise en place d'une base Postgres.....	11
3.1 Implémentation.....	11
3.2 Tests.....	12
Atelier 3B : Consumer via Confluent REST Proxy.....	13
3B.0 Mise en place de la base Postgres.....	13
3B.1 Découverte de l'API Consumer REST Proxy.....	13
3B.2 Script consumer Python fourni.....	14
3B.3 Tests de rééquilibrage.....	15
3B.4 Garanties de livraison.....	16
3B.5 Test de résilience broker.....	17
Atelier 4. Schema Registry et sérialisation Avro.....	18
4.1 Démarrage de Confuent Registry.....	18
4.2 Producteur de message.....	18
4.3 Consommateur de message.....	18
4.4 Evolution du schéma compatible.....	19
4.5 Evolution du schéma incompatible.....	19
Atelier 5. Kafka Connect.....	20
5.1 Installation du plugin ElasticSearch dans kafka connect.....	20
5.2 Démarrage de la stack.....	20
5.3 Configuration du connecteur.....	20
5.3 Améliorations.....	21
Atelier 6 : EOS.....	22
6.1. Transaction et ISOLATION_LEVEL.....	22
6.1.1 Producteur transactionnel.....	22
6.1.2 Consommateur.....	22
6.2 Transfert Exactly Once.....	22
6.3 Stockage et rétention.....	22
Atelier 7 : Introduction à KafkaStream.....	23
7.1 Opérateurs stateless.....	23
7.2 Opérateurs stateful.....	23
7.2.1 Count.....	23
7.2.2 Aggregate.....	23
Atelier 8 : ksqlDB.....	24
8.0 Démarrage de la stack.....	24
8.1 Getting Started.....	24
8.2 Transformations et filtres.....	25

8.3 Requêtes.....	26
8.4 Pour aller plus loin (optionnel).....	27
Ateliers 9: Sécurité.....	28
9.1 Mise en place de TLS.....	28
9.2 Authentification avec SASL/PLAIN.....	28
9.2.1 Authentification inter-broker.....	28
9.2.2 Authentification client.....	29
9.3 ACL.....	29
9.4 Quotas.....	29
Atelier 10 - Monitoring — Confluent Control Center.....	30
10.1 Explorer le cluster.....	30
10.2 Observer la production en temps réel.....	30
10.3 Observer le lag consommateur.....	31
10.4 Simuler une panne broker.....	31
10.5 Configurer une alerte.....	31

Atelier 1: Le cluster kafka

1.1 Premier démarrage et logs

Visualiser le fichier ***docker-compose.yml***

Il permet de démarrer un cluster 3 nœuds ainsi que 2 containers utiles pour l'administration du cluster

Démarrer cette stack avec :

```
docker compose up -d
```

Observer les logs de démarrages d'un nœud

```
docker logs -f kafka-1
```

1.2 Utilisation des utilitaires

Ouvrir un bash sur un des nœuds

```
docker exec -it kafka-1 bash
```

Créer un topic ***testing*** avec 5 partitions et 2 répliques :

```
kafka-topics --create --bootstrap-server kafka-1:9092 --replication-factor 2 --partitions 5 --topic testing
```

Lister les topics du cluster

Accéder à la description détaillée du topic

Démarrer un producteur de message

```
kafka-console-producer --bootstrap-server kafka-1:9092 --topic testing --property "parse.key=true" --property "key.separator=:"
```

Saisir quelques messages

Consommer les messages depuis le début

```
kafka-console-consumer --bootstrap-server kafka-1:9092 --topic testing --from-beginning
```

Dans une autre fenêtre, lister les groupes de consommateurs et accéder au détail du groupe de consommateur en lecture sur le topic ***testing***

1.3 Outils graphiques

Parcourir l'UI des 2 outils graphiques fournis :

Redpanda Console : <http://localhost:9090>

Confluent UI Center : <http://localhost:9021>

Atelier 2: Producer API (Java)

Importer le projet Maven fourni

Le projet est composé de :

- Une classe principale ***KafkaProducerApplication*** qui prend en arguments :
 - ***nbThreads*** : Un nombre de threads
 - ***nbMessages*** : Un nombre de messages
 - ***sleep*** : Un temps de pause en ms
 - ***sendMode*** : Le mode d'envoi : 0 pour Fire_And_Forget, 1 pour Synchrone, 2 pour AsynchroneL'application instancie *nbThreads KafkaProducerThread* et leur demande de s'exécuter ; quand toutes les threads sont terminées. Elle affiche le temps d'exécution
- Une classe ***KafkaProducerThread*** qui une fois instanciée envoie *nbMessages* tout les temps de pause selon un des 3 modes d'envoi.
Les messages sont constitués d'une au format String (*courier.id*) et d'une payload au format JSON (classe *Courier*)
- Le package ***model*** contient les classes modélisant les données
 - ***Position*** : Une position en latitude, longitude
 - ***Courier*** : Un coursier associé à une position
 - ***SendMode*** : Une énumération des modes d'envoi

Compléter les méthodes d'envoi de *KafkaProducerThread*.

Pour cela vous devez :

- Initialiser un *KafkaProducer<String,Courier>* et y positionner des sérialiseurs JSON pour la classe *Courier*
- Construire un *ProducerRecord* pour chaque messages
- Implémenter les 3 méthodes d'envoi

Tester l'envoi de messages dans les différents dans l'IDE

Via les commandes utilitaires ou l'interface d'admin, vérifier la création du topic et le contenu des messages

Construire un jar exécutable avec :

mvn package

Supprimer le topic et le recréer avec un nombre de *partitions=5* et un *replication-factor=3*

Envoyer 1M de messages, par exemple :

java -jar target/producer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 500 2000 10 1

Atelier 2B : Producer via Confluent REST Proxy

Objectifs

- Comprendre le rôle du Confluent REST Proxy comme passerelle HTTP → Kafka
- Envoyer des messages de positions de coursiers via des requêtes HTTP POST
- Observer l'impact des paramètres producer (acks, batch.size, linger.ms) sur les performances et la fiabilité
- Utiliser JMeter pour simuler plusieurs coursiers en parallèle et faire varier la charge

Contexte

Dans cet atelier, le rôle du client Kafka est joué par un outil de test de charge HTTP (Apache JMeter).

Chaque thread JMeter simule un coursier envoyant des positions GPS au topic `position` via le REST Proxy.

Le REST Proxy traduit chaque requête HTTP POST en un message Kafka Producer, en appliquant les paramètres de configuration que vous passerez dans les headers HTTP.

Le scénario métier est identique à l'atelier précédent : des coursiers envoient régulièrement leur position.

2B.1 Démarrage de la stack

Un fichier `docker-compose.yml` vous est fourni. Il étend le cluster 3 nœuds existant avec :

- ****Confluent REST Proxy**** (port 8082) : la passerelle HTTP → Kafka
- ****Redpanda Console**** reste disponible sur <http://localhost:9090>

Démarrer la stack complète :

`docker compose up -d`

Vérifier que le REST Proxy est opérationnel :

`curl http://localhost:8082/topics`

Vous devez obtenir la liste des topics existants au format JSON.

2B.2 Découverte du REST Proxy

Récupérer l'ID du cluster

`curl http://localhost:8082/v3/clusters`

Créer le topic `position` avec 5 partitions et un replication-factor de 3 via le REST Proxy :

```
curl -X POST http://localhost:8082/v3/clusters/[clusterId]/topics \
-H "Content-Type: application/json" \
-d '{
  "topic_name": "position",
  "partitions_count": 5,
  "replication_factor": 3
}'
```

Envoyer un message de test à la main pour valider la chaîne :

```
curl -X POST http://localhost:8082/topics/position \
-H "Content-Type: application/vnd.kafka.json.v2+json" \
-d '{
  "records": [
    {
      "key": "courier-1",
      "value": {
        "courierId": "courier-1",
        "latitude": 48.8566,
        "longitude": 2.3522
      }
    }
  ]
}'
```

Observer dans Redpanda Console (<http://localhost:9090>) la création du topic et la réception du message.

Identifier dans la réponse HTTP : l'offset, la partition et le topic de destination.

2B.3 Prise en main du script JMeter

Un script JMeter `kafka-rest-proxy.jmx` vous est fourni.

Lancer JMeter en mode graphique :

```
jmeter -t kafka-rest-proxy.jmx
```

Le script est organisé ainsi :

- ****Thread Group**** : simule les coursiers (1 thread = 1 coursier)
- ****HTTP Request**** : envoie un POST vers `http://localhost:8082/topics/position`
- ****HTTP Header Manager**** : configure le `Content-Type` et les headers Kafka (`X-Kafka-acks`, etc.)
- ****JSR223 PreProcessor**** : génère dynamiquement le payload JSON avec des coordonnées aléatoires et l'ID du coursier
- ****Aggregate Report**** : mesure les temps de réponse et le débit (throughput)
- ****Response Assertion**** : vérifie que chaque requête retourne un HTTP 200

Paramètres utilisateur (User Defined Variables)

En haut du plan de test, vous trouverez les variables suivantes, modifiables sans toucher au script :

- NB_COURSIERS : Nombre de threads (coursiers simultanés) (5)
- NB_MESSAGES : Nombre de messages par coursier (100)
- PAUSE_MS : Temps de pause entre deux messages (ms) (500)
- ACKS : Niveau de garantie : 0, 1 ou -1 (-1)
- BATCH_SIZE : Taille du batch en octets | 16384 |
- LINGER_MS : Délai de batching (ms) | 0 |

Premier test

Lancer le script avec les valeurs par défaut, observer dans Aggregate Report :

- Le throughput (messages/s)
- Les temps de réponse moyen, médian et P99

2B.4 Expérimentation des paramètres de fiabilité (acks)

Objectif

Mesurer l'impact du paramètre `acks` sur le débit et les temps de réponse.

Configuration

Fixer les variables communes pour avoir un test suffisamment long et stable :

- `NB\_COURSIERS` = 100
- `NB\_MESSAGES` = 5000
- `PAUSE\_MS` = 0 (envoi en rafale pour stresser)

Tests à effectuer

Exécuter 3 fois le test en changeant uniquement `ACKS` :

Test A — acks=0 (Fire and Forget)

- Aucun acquittement attendu
- Risque de perte de messages

Test B — acks=1 (Leader ACK)

- Seul le broker leader acquitte
- Compromis performance/fiabilité

Test C — acks=all (Full Durability)

- Tous les réplicas ISR acquittent
- Garantie maximale

Observations

Pour chaque test, noter dans un tableau :

- Throughput (msg/s)
- Temps de réponse moyen

Vérifier dans Redpanda Console le nombre de messages effectivement reçus dans le topic.

Avec `acks=0`, observer s'il y a des messages manquants sous charge.

2B.5 Expérimentation du batching (*batch.size* et *linger.ms*)

Objectif

Comprendre comment le batching améliore le débit en regroupant les messages avant envoi.

Rappel

- ***batch.size*** : taille maximum d'un batch en octets. Si le batch est plein, il est envoyé immédiatement.
- ***linger.ms*** : temps d'attente maximum avant d'envoyer un batch même s'il n'est pas plein. Augmenter cette valeur laisse le temps au batch de se remplir.

Ces deux paramètres interagissent : un batch est envoyé dès que *batch.size* est atteint ou que *linger.ms* est écoulé.

Configuration commune

- *NB_COURSIERS* = 100
- *NB_MESSAGES* = 5000
- *PAUSE_MS* = 0
- *ACKS* = 1

Tests à effectuer

Test 1 — Configuration par défaut

- *BATCH_SIZE* = 16384
- *LINGER_MS* = 0

Test 2 — Linger augmenté

- *BATCH_SIZE* = 16384
- *LINGER_MS* = 20

Test 3 — Batch et linger augmentés

- *BATCH_SIZE* = 65536
- *LINGER_MS* = 20

****Test 4 — Grand batch, fort linger****

- `BATCH\_SIZE` = 131072

- `LINGER\_MS` = 50

Observations

JMeter envoie les requêtes HTTP une par une. Il ne voit pas si le proxy les regroupe en un seul batch avant de les envoyer à Kafka. La latence HTTP individuelle peut même **augmenter** avec `linger.ms` élevé (le proxy attend avant d'envoyer) alors que le débit global côté Kafka s'améliore.

Dans Control Center, naviguer vers **Cluster** → **Brokers** → **Metrics** et observer :

- **Network Request Rate** : nombre de requêtes réseau reçues par les brokers. Avec un batching efficace, ce taux **baisse** car plusieurs messages sont regroupés en une seule requête réseau.
- **Bytes In/sec** : débit entrant brut — reste stable ou augmente car on envoie autant de données, juste en moins de requêtes.

2B.6 Arrêt d'un broker durant le test

Arrêter un broker pendant l'envoi :

```
docker stop kafka-2
```

Observer le comportement de JMeter : y a-t-il des erreurs ? Des retries ?

Redémarrer le broker :

```
docker compose up -d
```

Atelier 3 : Consumer API

3.0 Mise en place d'une base Postgres

Démarrer un serveur postgres et sa console d'administration

```
docker compose -f postgres-docker-compose.yml up -d
```

Accéder à *localhost:81* et se logger avec ***admin@admin.com/admin***

Enregistre un serveur avec comme paramètres de connexion :

- host : ***consumer-postgresql***
- user : ***postgres***
- password : ***postgres***

Créer une base consumer et y exécuter le script de création de table fourni : ***create-table.sql***

3.1 Implémentation

Le projet est composé de :

- Une classe principale ***KafkaConsumerApplication*** qui prend en arguments :
 - ***nbThreads*** : Un nombre de threads
L'application instancie *nbThreads KafkaConsumerThread* et leur demande de s'exécuter.
Le programme s'arrête au bout d'un certains temps.
- Une classe ***KafkaConsumerThread*** qui une fois instanciée poll le topic position
Le traitement d'un message consiste à un insérer un enregistrement en BD
- Le package ***model*** contient les classes modélisant les données
 - ***Position*** : Une position en latitude, longitude
 - ***Courier*** : Un coursier associé à une position

Compléter la boucle de réception des messages

Pour cela, vous devez

- Initialiser un *KafkaConsumer* avec la propriété ***ConsumerConfig.AUTO_OFFSET_RESET_CONFIG*** à « ***earliest*** »
- Fournir un Deserialiseur
- Implémenter la boucle de réception, pour chaque enregistrement reçu stocker dans la base postgres id du coursier et l'offset kafka en utilisant la classe ***ConsumerDao***

Pour tester la réception, vous pouvez utiliser le programme précédent et le lancer afin qu'il exécute de nombreux message :

Par exemple :

```
producer_home$ java -jar target/producer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 10 100000 500 0
```

Implémenter un ***ConsumerRebalanceListener*** et l'associer au moment du *subscribe()*

3.2 Tests

Une fois le programme mis au point, construire l'application avec :

```
mvn clean package
```

effectuer plusieurs tests, entre chaque démarrage effacer les offsets du groupe et nettoyer la base de données

Tester qu'aucun message n'est perdu :

Démarrer le programme avec 3 thread

```
java -jar target/consumer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 3 10
```

Visualiser la répartition des partitions

Arrêter puis redémarrer avec la même configuration

Tester la réaffectation de partitions :

Démarrer 2 fois l'application : 1 avec 2 threads l'autre avec 3 threads

```
java -jar target/consumer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 2 10
```

```
java -jar target/consumer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 3 10
```

visualiser la répartition des partitions

Arrêter et redémarrer un processus pendant la consommation

Test redémarrage broker

Arrêter un broker :

```
docker stop kafka-0
```

Redémarrer le, Dans le répertoire du fichier docker-compose

```
docker compose up -d
```

Essayer de vérifier le traitement de tous les messages. (Il se peut qu'il y ait des doublons de traitement)

Atelier 3B : Consumer via Confluent REST Proxy

Objectifs

- Consommer des messages Kafka via des requêtes HTTP GET sans client Kafka natif
- Comprendre le modèle stateful de la consommation REST Proxy (instance liée à un groupe)
- Observer le rééquilibrage des partitions en ajoutant ou supprimant des consommateurs
- Mesurer le lag de consommation dans Confluent Control Center
- Comprendre les contraintes d'affinité de session propres au REST Proxy

Contexte

Dans cet atelier, le consommateur est un script Python qui appelle l'API REST du proxy via des requêtes HTTP GET répétées. Le script insère chaque message reçu dans une base Postgres — même modèle de données que l'atelier Java original.

Le producteur de messages est le script JMeter de l'atelier précédent.

3B.0 Mise en place de la base Postgres

Démarrer le serveur Postgres et sa console d'administration :

```
docker compose -f postgres-docker-compose.yml up -d
```

Accéder à pgAdmin sur `http://localhost:81` avec `admin@admin.com / admin`.

Enregistrer un serveur avec les paramètres :

- `host : consumer-postgresql`
- `user : postgres`
- `password : postgres`

Créer une base `consumer` et y exécuter le script `create-table.sql` fourni.

3B.1 Découverte de l'API Consumer REST Proxy

La consommation via REST Proxy est **stateful** : contrairement à la production, il faut d'abord créer une instance de consommateur sur le proxy, puis appeler régulièrement `GET /records` pour dépiler les messages.

Contrainte importante : toutes les requêtes d'une instance consommateur doivent aller vers **la même instance du proxy**. Dans cet atelier avec un proxy unique, ce n'est pas un problème. En production avec plusieurs instances proxy, un load balancer avec session affinity (sticky sessions) est obligatoire.

Étape 1 — Créer l'instance consommateur

```
curl -X POST http://localhost:8082/consumers/position-group \  
-H "Content-Type: application/vnd.kafka.v2+json" \  
-d '{  
  "name": "consumer-1",  
  "format": "json",  
  "auto.offset.reset": "earliest",  
  "auto.commit.enable": "false"
```

```
}'
```

La réponse contient le `base_uri` à utiliser pour toutes les requêtes suivantes :

```
{
  "instance_id": "consumer-1",
  "base_uri":
"http://localhost:8082/consumers/position-group/instances/consumer-1"
}
```

Stocker ce `base_uri` dans une variable :

```
BASE_URI="http://localhost:8082/consumers/position-group/instances/consumer-1"
```

Étape 2 — S'abonner au topic

```
curl -X POST ${BASE_URI}/subscription \
  -H "Content-Type: application/vnd.kafka.v2+json" \
  -d '{"topics": ["position"]}'
```

Étape 3 — Consommer des messages (appel à répéter)

```
curl -X GET ${BASE_URI}/records \
  -H "Accept: application/vnd.kafka.json.v2+json"
```

La réponse est un tableau JSON de messages. Si le topic est vide ou si aucun message n'est disponible, la réponse est un tableau vide `[]`.

Pourquoi répéter l'appel ? Contrairement au client Java natif qui maintient une connexion TCP persistante, le REST Proxy ne pousse pas les messages — c'est le client qui poll. Il faut donc appeler `GET /records` en boucle.

Étape 4 — Commiter les offsets

```
curl -X POST ${BASE_URI}/offsets \
  -H "Content-Type: application/vnd.kafka.v2+json" \
  -d '{
    "offsets": [
      {"topic": "position", "partition": 0, "offset": 10},
      {"topic": "position", "partition": 1, "offset": 5}
    ]
  }'
```

Étape 5 — Supprimer l'instance en fin de session

```
curl -X DELETE ${BASE_URI} \
  -H "Content-Type: application/vnd.kafka.v2+json"
```

Ne pas supprimer l'instance → elle reste active sur le proxy jusqu'au timeout (`consumer.instance.timeout.ms`, défaut 5 minutes).

3B.2 Script consumer Python fourni

Un script **consumer-rest.py** vous est fourni. Il encapsule les 5 étapes ci-dessus dans une boucle continue et insère chaque message reçu en base Postgres.

Structure du script

Le script prend les arguments suivants :

Argument	Description	Défaut
--group	Nom du consumer group	position-group
--consumer-id	Identifiant unique de l'instance	consumer-1
--proxy	URL du REST Proxy	http://localhost:8082
--topic	Topic à consommer	position
--poll-interval	Délai entre deux GET /records (ms)	500

Éventuellement, ajouter les modules supplémentaires :

```
python3 -m venv ~/kafka-venv
source ~/kafka-venv/bin/activate
pip install requests psycpg2-binary
```

Lancer le script

```
python3 consumer-rest.py --consumer-id consumer-1
```

Observer dans le terminal :

- Les messages reçus à chaque appel GET
- Les offsets commités après chaque batch
- Les insertions en base Postgres

Vérifier en parallèle dans pgAdmin que les enregistrements apparaissent dans la table.

3B.3 Tests de rééquilibrage

Objectif

Observer comment le proxy redistribue les partitions quand des consommateurs rejoignent ou quittent le groupe.

Démarrer le producteur

Depuis le script JMeter (atelier 2), lancer un envoi continu :

- NB_COURSIERS = 20
- NB_MESSAGES = 10000
- PAUSE_MS = 100
- ACKS = 1

Test A — Un seul consommateur

Démarrer un premier consommateur dans un terminal :

```
python3 consumer-rest.py --consumer-id consumer-1
```

Dans **Confluent Control Center** (<http://localhost:9021>):

- Naviguer vers Consumer Groups → position-group
- Observer que le consommateur a reçu les 5 partitions du topic position
- Relever le lag de consommation

Test B — Ajout d'un deuxième consommateur

Ouvrir un deuxième terminal et démarrer un second consommateur :

```
python3 consumer-rest.py --consumer-id consumer-2
```


Dans Control Center, observer **le rééquilibrage** :

- Les 5 partitions se redistribuent entre les 2 consommateurs
- Le lag évolue-t-il ? À quelle vitesse est-il résorbé ?

Questions :

- Combien de partitions chaque consommateur reçoit-il ?
- Y a-t-il une pause dans la consommation pendant le rééquilibrage ?

Test C — 5 consommateurs (parallélisme optimal)

Ouvrir 3 terminaux supplémentaires et démarrer consumer-3, consumer-4, consumer-5.

Observer dans Control Center :

- La répartition 1 partition / 1 consommateur
- Le lag se résorbe-t-il plus vite ?

Test D — 6 consommateurs (au-delà du nombre de partitions)

Démarrer un 6e consommateur :

```
python3 consumer-rest.py --consumer-id consumer-6
```

Observer dans Control Center :

- Le 6e consommateur reçoit-il des partitions ?
- Que se passe-t-il dans le terminal du consumer-6 ?

Test E — Arrêt d'un consommateur

Arrêter consumer-2 avec Ctrl+C.

Observer dans Control Center :

- Le rééquilibrage se déclenche-t-il automatiquement ?
- Dans quel délai ? (lié au `session.timeout.ms`)
- Les partitions de consumer-2 sont-elles redistribuées ?

Différence avec l'API Java : le rééquilibrage est géré côté proxy, pas côté client HTTP.
Le script Python ne reçoit pas de callback `onPartitionsRevoked` / `onPartitionsAssigned` — c'est transparent.

3B.4 Garanties de livraison

Objectif

Observer le comportement At Least Once avec commit manuel.

Préparer le test

Vider la table Postgres et réinitialiser les offsets du groupe :

```
# Depuis le container kafka-1
docker exec -it kafka-1 \
  kafka-consumer-groups --bootstrap-server kafka-1:9092 \
  --group position-group --reset-offsets \
  --to-earliest --topic position --execute
```

Test At Least Once

Lancer le producteur JMeter avec 1 000 messages, puis démarrer un consommateur en provoquant un crash :

```
python3 consumer-rest.py --consumer-id consumer-1 --crash-after 50
```

Relancer le consommateur avec le **même consumer-id** et le **même group** :

```
python3 consumer-rest.py --consumer-id consumer-1
```

Observer :

- Le consommateur reprend-il depuis le dernier offset commité ?

3B.5 Test de résilience broker

Arrêt d'un broker pendant la consommation

Lancer JMeter en continu et démarrer un consommateur, puis arrêter un broker :

```
docker stop kafka-2
```

Observer :

- Dans le terminal du consumer : des erreurs apparaissent-elles ?
- Dans Control Center : le cluster réélect-il un nouveau leader pour les partitions affectées ?
- La consommation reprend-elle automatiquement ?

Redémarrer le broker :

```
docker compose -f docker-compose-rest.yml up -d
```

Observer la réintégration dans Control Center.

Atelier 4. Schema Registry et sérialisation Avro

4.1 Démarrage de Confluent Registry

Visualiser le fichier ***docker-compose.yml*** fourni qui démarre Redpanda Console, akhq et SchemaRegistry de confluent

Démarrer la stack via :

```
docker compose up -d
```

Vérifier le bon démarrage des containers

Accéder à *localhost:8085/subjects*

4.2 Producteur de message

Récupérer le projet ***producer-avro***

Visualiser le schéma Avro :

src/main/resources/Courier.avsc

Lancer la commande ***./mvnw compile*** et regarder les classes générées par le plugin Avro

Lancer ensuite ***./mvnw package*** pour construire l'exécutable

Démarrer l'exécutable par

```
java -jar target/producer-avro-0.0.1-SNAPSHOT-jar-with-dependencies.jar 10 500 10 0
```

Le programme, après avoir enregistré le schéma, envoie 10*500 messages qui sont sérialisés avec Avro sur le topic ***avro-position***

Après un premier démarrage, le schéma doit être consultable :

- sur schema-registry (localhost:8085/subjects)
- ou via votre outil d'admin

Vérifier que l'outil graphique puisse lire les messages Avro

4.3 Consommateur de message

Récupérer le projet ***consumer-avro***

Visualiser le code source et en particulier la boucle de poll et l'utilisation de ***GenericMessage***

Lancer ensuite ***./mvnw package*** pour construire l'exécutable

Consommer les messages du topic précédent avec la commande

```
java -jar target/consumer-0.0.1-SNAPSHOT-jar-with-dependencies.jar 2 1
```

Laisser le programme tourner

4.4 Evolution du schéma compatible

Mettre à jour le schéma en ajoutant les champs optionnels : **firstName** dans la structure **Coursier**

```
{
    "name": "first_name",
    "type": "string",
    "default": "undefined"
},
```

Fixer les problèmes de compilation :

Dans **src/main/java/org/formation/KafkaProducerThread.java**

Changer la ligne 29 par :

```
this.courier = new Courier(id, "David", new Position(Math.random() + 45,
Math.random() + 2));
```

Reconstruire l'application via :

./mvnw clean package

Relancer le programme de production

java -jar target/producer-avro-0.0.1-SNAPSHOT-jar-with-dependencies.jar 10 500 10 0

Visualiser ensuite la nouvelle version du schéma dans le registre

Consommer les messages sans modifications du programme consommateur

4.5 Evolution du schéma incompatible

Mettre à jour le schéma en ajoutant un champs obligatoire : **vehicle_id** dans la structure **Coursier**

```
{
    "name": "vehicle_id",
    "type": "int"
},
```

Fixer les problèmes de compilation

Dans **src/main/java/org/formation/KafkaProducerThread.java**

Changer la ligne 29 par :

```
this.courier = new Courier(id, "David", 1, new Position(Math.random() + 45,
Math.random() + 2));
```

Relancer le programme de production et visualiser l'exception au moment de l'enregistrement du nouveau schéma.

Atelier 5. Kafka Connect

Objectifs : Déverser le topic *position* dans un index ElasticSearch

5.1 Installation du plugin ElasticSearch dans kafka connect

Visualiser le fichier Dockerfile fourni et construire une image *my-kafka-connect-with-elasticsearch* via :

```
docker build -t my-kafka-connect-with-elasticsearch .
```

5.2 Démarrage de la stack

Démarrer la nouvelle stack qui ajoute l'image précédemment construite, ElasticSearch et Kibana

```
docker-compose up -d
```

Vérifier l'installation du plugin via :

<http://localhost:8083/connector-plugins>

Se connecter à kibana localhost :5601 et dans la DevConsole exécuter :

```
PUT /position
```

```
{
  "mappings":{
    "properties": {
      "@timestamp": {
        "type": "date",
        "format": "epoch_millis"
      }
    }
  }
}
```

La commande crée un index elasticsearch *position* avec pour l'instant un seul champ @timestamp

5.3 Configuration du connecteur

Définir un connecteur *elasticsearch-sink* qui déverse le topic position dans l'index position :

```
curl -X POST http://localhost:8083/connectors \
-H "Content-Type: application/json" \
-d '{
"name": "elasticsearch-sink",
"config": {
"connector.class":
```

```
"io.confluent.connect.elasticsearch.ElasticsearchSinkConnector",
"tasks.max": "1",
"topics": "position",
"topic.index.map": "position:position_index",
"connection.url": "http://elasticsearch:9200",
"type.name": "log",
"key.ignore": "true",
"schema.ignore": "true",
"key.converter": "org.apache.kafka.connect.json.JsonConverter",
"key.converter.schemas.enable": "false",
"value.converter": "org.apache.kafka.connect.json.JsonConverter",
"value.converter.schemas.enable": "false"
}
}'
```

Vous pouvez vérifier la bonne installation du connecteur :

- Via les logs de KafkaConnect
- Via [http://localhost:8083 /connectors](http://localhost:8083/connectors) : Liste des connecteurs actifs
- Via l'interface de RedPanda

Alimenter le topic position

Vous pouvez visualiser les effets du connecteur

- Via ElasticSearch : http://localhost:9200/position/_search
- Via Kibana : <http://localhost:5601>

5.3 Améliorations

Améliorer le fichier de configuration afin que le timestamp Kafka soit renseigné dans le champ **@timestamp** de l'index d'ElasticSearch

Créer un autre connecteur pour déverser le topic **position-avro** dans un index **position-avro**

Atelier 6 : EOS

6.1. Transaction et ISOLATION_LEVEL

6.1.1 Producteur transactionnel

Modifier le code du producer afin d'englober plusieurs envois de messages dans une transaction.

Commiter tous les 10 envois

Lancer ensuite le programme avec comme arguments 1 **105** 100 0

Ce qui signifie que 1 threads envoie 105 messages, les 100 premiers messages font partie de 10 transactions validées. Les 5 derniers messages ne sont pas validés.

Visualiser les messages dans une console d'administration.

Noter le nombre et les offsets, Visualiser le flag transactionnel

6.1.2 Consommateur

Modifier la configuration du consommateur afin qu'il ne lise que les messages committés

6.2 Transfert Exactly Once

Modifier le consommateur afin qu'il transfère exactement une fois les messages produits en amont vers un autre topic de sortie

Éventuellement on peut effectuer un traitement du message comme calculer la distance à un point d'origine.

Vérifier les messages produits et les offsets consommés

6.3 Stockage et rétention

Ouvrir un bash sur un des nœuds

docker exec -it kafka-1 bash

Visualiser les répertoires de logs sur les brokers :

kafka-log-dirs.sh --bootstrap-server localhost:9092 --describe

Visualiser les segments du topic position et apprécier la taille

Pour le topic position modifier le **segment.bytes** à 10Mo

Diminuer le **retention.bytes** à 50Mo

Produire de nombreux messages afin de voir des segments disparaître

Atelier 7 : Introduction à KafkaStream

Objectifs :

Écrire une application Stream qui prend en entrée le topic *position-avro*

7.1 Opérateurs stateless

Créer un projet *PositionStream* et installer le package *io.confluent :kafka-avro-serializer*

Reprendre le schéma Avro initial du producteur Avro et générer les classes du modèle

- Dans un premier temps, transformer les informations de la position d'un coursier en arrondissant la longitude et la latitude à 1 valeur entière.

Tester, regarder les timestamp des messages

Supprimer le topic de sortie

- Compléter le stream en inversant les clés et Valeurs (La position du coursier devient la clé, l'id du coursier la valeur)

Tester

- Utiliser branch pour créer 2 topics de sortie un contenant les positions dont la latitude est supérieure à 45.0 et l'autre le reste

Tester

7.2 Opérateurs stateful

7.2.1 Count

Sur les branches nord/sud précédentes, effectuer une agrégation de type Count(), i.e compter le nombre de coursier ayant passé à une position donné.

Tester

Modifier en utilisant une fenêtre temporelle de 5 minutes.

7.2.2 Aggregate

Nous voulons dorénavant avoir en temps-réel, la liste des coursiers associés à une position.

Implémenter via une fonction aggregate, la valeur agrégée étant une liste

- Partir du topic position
- Transformer la valeur en Position en arrondissant la latitude et la longitude à une position entière
- Transformer en une table contenant la dernière Position du Coursier
- Grouper la table par Position
- Agréger en utilisant un adder et subtractor

Atelier 8 : ksqlDB

Objectifs

- Découvrir l'interface CLI de ksqlDB
- Créer des streams et des tables à partir du topic `position` produit aux ateliers précédents
- Écrire des requêtes dérivées pour filtrer et agréger les positions des coursiers
- Comprendre la différence entre Push Query (temps réel) et Pull Query (snapshot)

Contexte

Dans cet atelier, ksqlDB consomme directement le topic `position` produit par JMeter (atelier 2). Aucun code Java n'est nécessaire — tout se fait en SQL depuis la CLI ou via l'API REST.

8.0 Démarrage de la stack

Un fichier `docker-compose-ksqldb.yml` vous est fourni. Il étend la stack existante avec :

- **ksqlDB Server** (port 8088)
- **ksqlDB CLI** (client interactif)

```
docker compose -f docker-compose-ksqldb.yml up -d
```

Vérifier que le serveur est opérationnel :

```
curl http://localhost:8088/info
```

8.1 Getting Started

Connexion à la CLI

```
docker exec -it ksqldb-cli ksql http://ksqldb-server:8088
```

Explorer l'environnement :

```
SHOW TOPICS;  
SHOW STREAMS;  
SHOW TABLES;
```

Inspecter le topic `position` pour voir les messages existants :

```
PRINT 'position' FROM BEGINNING LIMIT 5;
```

Observer la structure d'un message : `courierId`, `latitude`, `longitude`, `timestamp`.

Créer le stream de base

Créer un stream `positions` mappé sur le topic `position` :

```
CREATE STREAM positions (  
  courierId VARCHAR,  
  latitude DOUBLE,  
  longitude DOUBLE  
) WITH (  
  kafka_topic = 'position',  
  value_format = 'JSON'  
);
```

Vérifier la création :

```
DESCRIBE positions;
```

Exécuter une première push query pour voir les messages en temps réel. **Dans un autre terminal**, démarrer JMeter avec NB_COURSIERS=5, NB_MESSAGES=100, PAUSE_MS=200.

```
SELECT courierId, latitude, longitude
FROM positions
EMIT CHANGES
LIMIT 10;
```

Observer les messages défiler. Appuyer sur Ctrl+C pour arrêter.

8.2 Transformations et filtres

Stream dérivé — positions dans Paris

Créer un stream ne contenant que les positions dans la zone parisienne (latitude entre 48.8 et 48.95, longitude entre 2.2 et 2.5) :

```
CREATE STREAM positions_paris AS
  SELECT courierId,
         latitude,
         longitude
  FROM positions
  WHERE latitude BETWEEN 48.80 AND 48.95
         AND longitude BETWEEN 2.20 AND 2.50
  EMIT CHANGES;
```

Vérifier que le topic correspondant a été créé :

```
SHOW TOPICS;
```

Constater que ksqlDB a créé automatiquement le topic POSITIONS_PARIS.

Exécuter une push query sur ce stream filtré :

```
SELECT * FROM positions_paris EMIT CHANGES LIMIT 5;
```

Table — dernière position connue par coursier

Créer une table qui maintient en temps réel la **dernière position connue** de chaque coursier :

```
CREATE TABLE last_position AS
  SELECT courierId,
         LATEST_BY_OFFSET(latitude) AS last_lat,
         LATEST_BY_OFFSET(longitude) AS last_lon,
         COUNT(*) AS nb_messages
  FROM positions
  GROUP BY courierId
  EMIT CHANGES;
```

La table est mise à jour à chaque nouveau message reçu pour un coursier donné.

Agrégation par zone — coursiers par secteur

Arrondir les coordonnées à 1 décimale pour créer des secteurs géographiques, et compter combien de coursiers différents ont été vus dans chaque secteur :

```
CREATE TABLE positions_par_secteur AS
  SELECT CAST(ROUND(last_lat, 1) AS VARCHAR) + '_' +
```

```

        CAST(ROUND(last_lon, 1) AS VARCHAR) AS secteur,
        COUNT(*) AS nb_coursiers,
        COLLECT_LIST(courierId) AS coursiers
    FROM last_position
    GROUP BY CAST(ROUND(last_lat, 1) AS VARCHAR) + '_' +
        CAST(ROUND(last_lon, 1) AS VARCHAR)
    EMIT CHANGES;

```

Faire une sélection sur la table

8.3 Requêtes

Push Query — suivi en temps réel

Lancer JMeter avec NB_COURSIERS=10, NB_MESSAGES=1000, PAUSE_MS=100.

Dans la CLI, suivre en temps réel les mises à jour de la table `last_position` :

```

SELECT courierId, last_lat, last_lon, nb_messages
FROM last_position
EMIT CHANGES;

```

Observer les lignes se mettre à jour au fil des messages produits.

Filtrer sur un seul coursier (remplacer `courier-0` par un ID visible dans les logs) :

```

SELECT *
FROM last_position
WHERE courierId = 'courier-0'
EMIT CHANGES;

```

Pull Query — snapshot à l'instant T

Une fois JMeter arrêté, interroger la table pour obtenir l'état courant :

```

-- Dernière position de tous les coursiers
SELECT courierId, last_lat, last_lon, nb_messages
FROM last_position;

```

```

-- Coursier ayant envoyé le plus de messages
SELECT courierId, nb_messages
FROM last_position
WHERE nb_messages > 50;

```

Pull Query vs Push Query : la pull query retourne un snapshot et se termine immédiatement. La push query reste ouverte et pousse les mises à jour en continu.

Push Query avec fenêtre temporelle

Compter les messages par coursier sur des fenêtres glissantes de 1 minute :

```

SELECT courierId,
        WINDOWSTART AS debut,
        WINDOWEND AS fin,
        COUNT(*) AS nb
FROM positions
WINDOW TUMBLING (SIZE 1 MINUTE)
GROUP BY courierId
EMIT CHANGES;

```

Relancer JMeter et observer les compteurs se réinitialiser à chaque nouvelle fenêtre.

8.4 Pour aller plus loin (optionnel)

Requête de proximité avec GEO_DISTANCE

Créer une table des coursiers proches de la Place de la République (48.8672, 2.3628) :

```
CREATE TABLE coursiers_proche_republique AS
  SELECT courierId,
         last_lat,
         last_lon,
         ROUND(GEO_DISTANCE(last_lat, last_lon, 48.8672, 2.3628), 2) AS distance_km
  FROM last_position
 WHERE GEO_DISTANCE(last_lat, last_lon, 48.8672, 2.3628) <= 5.0
EMIT CHANGES;
```

Pull query pour voir les coursiers actuellement proches :

```
SELECT * FROM coursiers_proche_republique;
```

Explorer les objets créés

```
DESCRIBE EXTENDED last_position;
SHOW QUERIES;
EXPLAIN <query_id>;
```

DESCRIBE EXTENDED affiche les statistiques d'exécution de la requête sous-jacente. SHOW QUERIES liste toutes les requêtes persistantes en cours d'exécution.

Nettoyage

```
DROP TABLE coursiers_proche_republique DELETE TOPIC;
DROP TABLE positions_par_secteur DELETE TOPIC;
DROP TABLE last_position DELETE TOPIC;
DROP STREAM positions_paris DELETE TOPIC;
DROP STREAM positions;
```

Ateliers 9: Sécurité

9.1 Mise en place de TLS

Redémarrer le cluster en utilisant la nouvelle version de docker-compose, le port EXTERNAL utilise dorénavant SSL

Récupérer le répertoire SSL qui contient un keystore et un trustore self-signed pour localhost

Modifier **docker-compose.yml** afin que le répertoire **ssl/mount** soit correctement monté sur chaque broker.

Démarrer le cluster et vérifier le certificat produit par le serveur :

```
openssl s_client -debug -connect localhost:19092
```

Modifier les propriétés du projet producer afin qu'il utilise le protocole SSL pour communiquer

```
kafkaProps.put("security.protocol", "SSL");  
kafkaProps.put("ssl.truststore.location",  
"path-to/kafka.truststore.jks");  
kafkaProps.put("ssl.truststore.password", "secret");
```

Vérifier la production de message

9.2 Authentification avec SASL/PLAIN

9.2.1 Authentification inter-broker

Mettre au point un fichier **kafka_jaas.conf** définissant 2 utilisateurs *admin* et *alice* et indiquant que le serveur utilise l'identité *admin* comme suit :

```
KafkaServer {  
org.apache.kafka.common.security.plain.PlainLoginModule required  
username="admin"  
password="admin-secret"  
user_admin="admin-secret"  
user_rest-proxy="rest-proxy-secret"  
user_alice="alice-secret"  
user_bob="bob-secret";  
};
```

Récupérer la nouvelle version de docker-compose et vérifier le montage de volume vers votre fichier **kafka_jaas.conf**

Vérifier également la configuration du **rest-proxy**

Redémarrer le cluster et vérifier son bon démarrage

9.2.2 Authentication client

Mettre à jour un fichier **client-ssl.properties** comme suit :

```
security.protocol=SASL_SSL
```

```
sasl.mechanism=PLAIN
```

```
sasl.jaas.config=org.apache.kafka.common.security.plain.PlainLoginModule required \
```

```
  username="alice" \
```

```
  password="alice-secret";
```

Tester avec :

```
docker exec -it kafka-1 bash
```

```
kafka-topics.sh --bootstrap-server localhost:19092 --list --command-config  
ssl/client-ssl.properties
```

Configurer le projet producer pour qu'il s'authentifie avec l'utilisateur alice

Tester l'envoi de message, vérifier la bonne configuration du *KafkaProducer* et la production de message

9.3 ACL

Visualiser la nouvelle configuration des brokers dans le nouveau **docker-compose.yml**

Démarrer le cluster.

Vérifier via Redpanda Console que le lien sécurité permet de créer des ACLs

Interdire à Alice la production de message et tester

9.4 Quotas

Définition d'un quota pour l'utilisateur *alice*

Tester avec :

```
docker exec -it kafka-1 bash
```

```
kafka-configs.sh --bootstrap-server localhost:9094 --alter --add-config  
'producer_byte_rate=1024,consumer_byte_rate=1024' --entity-type users --entity-  
name alice
```

Relancer producer et observer les ralentissements de débit

Supprimer les quotas :

```
kafka-configs.sh --bootstrap-server localhost:9092 --alter --delete-config  
producer_byte_rate --entity-type users --entity-name alice
```

```
kafka-configs.sh --bootstrap-server localhost:9092 --alter --delete-config  
consumer_byte_rate --entity-type users --entity-name alice
```

Atelier 10 - Monitoring — Confluent Control Center

Objectifs

- Naviguer dans Confluent Control Center
- Observer le débit de production en temps réel
- Mesurer et interpréter le lag d'un consumer group
- Configurer une alerte et la déclencher

Prérequis

La stack avec Control Center doit être démarrée (`docker-compose up -d`). Control Center est accessible sur `http://localhost:9021`.

Le topic `position` doit exister avec 5 partitions (créé à l'atelier 2).

10.1 Explorer le cluster

Accéder à `http://localhost:9021` et naviguer vers **Cluster** → **Overview**.

Observer :

- Le nombre de brokers actifs
- Le graphique **Messages/sec** (doit être à 0 pour l'instant)
- La valeur **Under-Replicated Partitions** — doit être à 0

Naviguer vers **Cluster** → **Brokers** et relever pour chaque broker :

- Le **Network Request Rate**
- Les **Bytes In/sec** et **Bytes Out/sec**

Naviguer vers **Cluster** → **Topics** → **position** et noter :

- Le nombre de partitions
- Le facteur de réplication
- La répartition des leaders entre les brokers

10.2 Observer la production en temps réel

Lancer JMeter avec les paramètres suivants :

- `NB_COURSIERS = 20`
- `NB_MESSAGES = 10000`
- `PAUSE_MS = 50`
- `ACKS = 1`

Dans Control Center, naviguer vers **Topics** → **position** → **Overview**.

Observer en temps réel :

- Le graphique **Messages/sec produits**
- La répartition des messages entre les 5 partitions

Naviguer vers **Cluster** → **Brokers** → **Metrics** et observer :

- Le **Network Request Rate** augmenter
- Les **Bytes In/sec** sur chaque broker

Question : la charge est-elle équilibrée entre les 3 brokers ?

10.3 Observer le lag consommateur

Pendant que JMeter produit, démarrer le script consommateur dans un terminal :

```
python3 consumer-rest.py --consumer-id consumer-1 --poll-interval 1000
```

Dans Control Center, naviguer vers **Cluster** → **Consumer Groups** → **position-group**.

Observer :

- Le lag total du groupe
- Le lag par partition — est-il équilibré ?
- La colonne **CONSUMER-ID** — quel membre est assigné à quelle partition

Provoquer un lag croissant en ralentissant le consommateur :

Arrêter le script et le relancer avec un poll interval plus long :

```
python3 consumer-rest.py --consumer-id consumer-1 --poll-interval 3000
```

Observer dans Control Center le lag qui augmente sur le graphique d'évolution.

Relancer JMeter si nécessaire pour maintenir la production.

Question : à partir de quel délai le lag commence-t-il à croître ?

10.4 Simuler une panne broker

Pendant que JMeter produit, arrêter un broker :

```
docker stop kafka-2
```

Dans Control Center, observer immédiatement :

- **Under-Replicated Partitions** passe à une valeur > 0
- **Broker Count** passe à 2
- Les leaders de partitions sont réélus sur les brokers restants

Relancer le broker :

```
docker compose up -d kafka-2
```

Observer la réintégration progressive du broker et le retour à 0 des Under-Replicated Partitions.

10.5 Configurer une alerte

Dans Control Center, naviguer vers **Menu** → **Alerts** → + **New alert**.

Configurer l'alerte suivante :

Champ	Valeur
Metric	Under-Replicated Partitions
Condition	> 0
Duration	30 secondes
Notification	(email ou webhook si disponible)

Déclencher l'alerte en arrêtant à nouveau un broker :

```
docker stop kafka-2
```

Vérifier dans **Alerts** → **History** que l'alerte est bien déclenchée au bout de 30 secondes.

Relancer le broker et observer la résolution de l'alerte.